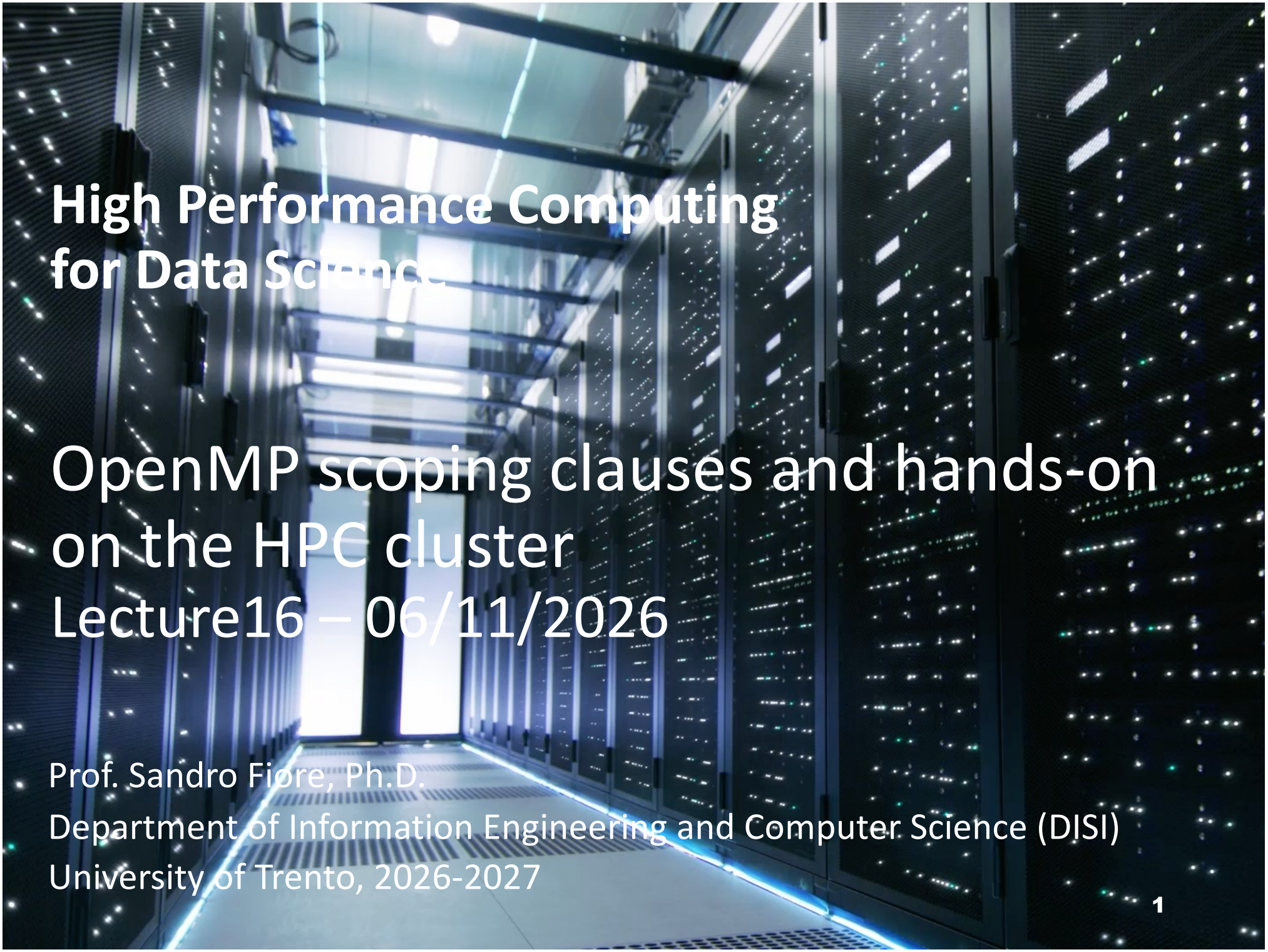


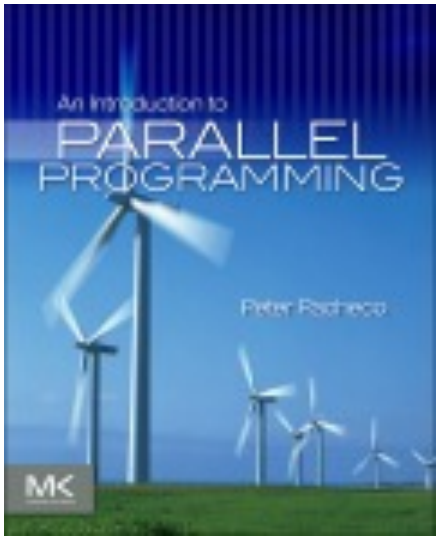
High Performance Computing for Data Science

OpenMP scoping clauses and hands-on on the HPC cluster
Lecture16 – 06/11/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Outline and material

- Parallel for and scoping clauses (recap)
- Scoping rules exercise
- Practical hands-on with OpenMP



Material

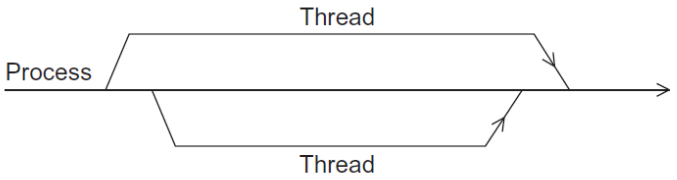
Book
An Introduction to Parallel Programming by Peter Pacheco

Chapter 5
Shared Memory Programming with OpenMP

Parallel and Parallel for

Parallel

- Most basic parallel directive.
- The number of threads that run the following structured block of code is determined by the run-time system.



Parallel for

- **Forks a team of threads** to execute the following structured block.
- However, **the structured block** following the parallel for directive **must be a for loop**.
- Furthermore, with the *parallel for directive* the system *parallelizes the for loop by dividing the iterations* of the loop *among the threads*.

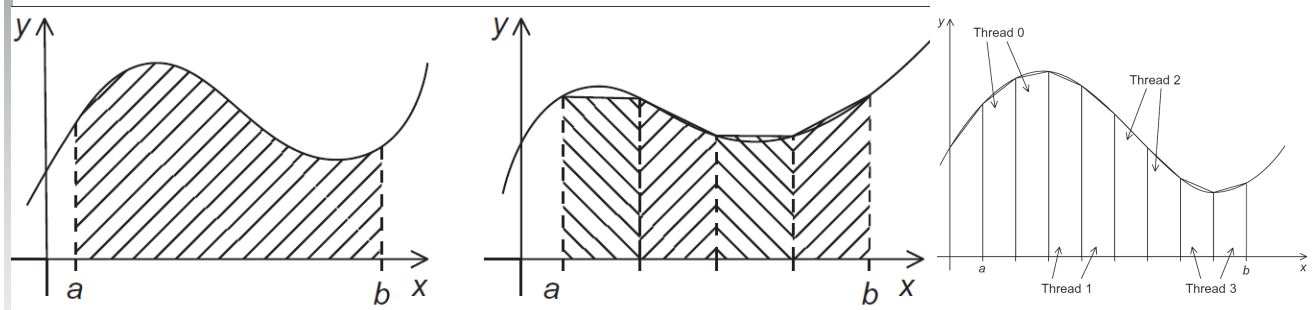
Copyright © 2010, Elsevier Inc. All rights Reserved

Parallel for: caveats

- The variable index must have integer or pointer type (e.g., it can't be a float).
- The expressions start, end, and incr must have a compatible type. For example, if index is a pointer, then incr must have integer type.
- The expressions start, end, and incr must not change during execution of the loop.
- During execution of the loop, the variable index can only be modified by the "increment expression" in the for statement.

Copyright © 2010, Elsevier Inc. All rights Reserved

Trapezoidal rule OpenMP (I)




```
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++)
   approx += f(a + i*h);
approx = h*approx;
```

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
   double  global_result = 0.0;  /* Store result in global_result */
   double  a, b;                 /* Left and right endpoints      */
   int     n;                    /* Total number of trapezoids    */
   int     thread_count;

   thread_count = strtol(argv[1], NULL, 10);
   printf("Enter a, b, and n\n");
   scanf("%lf %lf %d", &a, &b, &n);
#  pragma omp parallel num_threads(thread_count)
   Trap(a, b, n, &global_result);

   printf("With n = %d trapezoids, our estimate\n", n);
   printf("of the integral from %f to %f = %.14e\n",
      a, b, global_result);
   return 0;
}  /* main */
```

MPI style

```
void Trap(double a, double b, int n, double* global_result_p) {
   double  h, x, my_result;
   double  local_a, local_b;
   int  i, local_n;
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

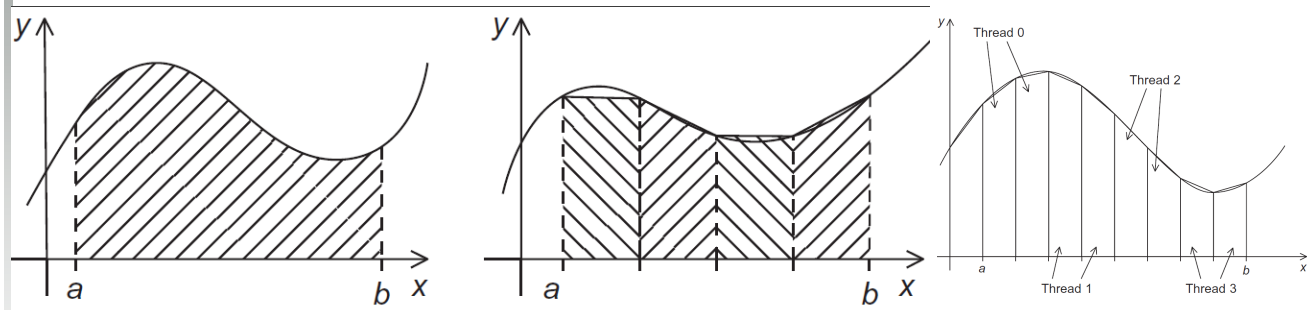
   h = (b-a)/n;
   local_n = n/thread_count;
   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   my_result = (f(local_a) + f(local_b))/2.0;
   for (i = 1; i <= local_n-1; i++) {
     x = local_a + i*h;
     my_result += f(x);
   }
   my_result = my_result*h;

#  pragma omp critical
   *global_result_p += my_result;
}  /* Trap */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Trapezoidal rule OpenMP (II)




```
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++)
   approx += f(a + i*h);
approx = h*approx;
```

```
   h = (b-a)/n;
   approx = (f(a) + f(b))/2.0;
#  pragma omp parallel for num_threads(thread_count) \
      reduction(+: approx)
   for (i = 1; i <= n-1; i++)
      approx += f(a + i*h);
   approx = h*approx;
```

OpenMP style

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Scoping clauses list

- **default**
- **shared**
- **private**
- **reduction**
- firstprivate
- lastprivate

Scope in OpenMP

- **Scoping clauses**: shared, private, reduction, default, firstprivate, lastprivate
  - A variable that can be accessed by all the threads in the team has shared scope.
    - The default scope for variables declared before a parallel block is shared.
  - A variable that can only be accessed by a single thread has private scope.
  - default: lets the programmer specify the scope of each variable in a block.
    - *default (none)* → the compiler will require that we specify the scope of each variable we use in the block and that has been declared outside the block.

Copyright © 2010, Elsevier Inc. All rights Reserved

Reduction operators

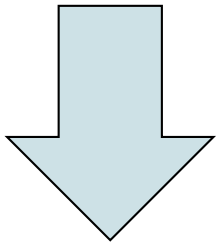
- A reduction operator is a binary operation (such as addition or multiplication).
- A reduction is a computation that repeatedly applies the same reduction operator to a sequence of operands in order to get a single result.
- All of the intermediate results of the operation should be stored in the same variable: the ***reduction variable***.
  - Double scope! Private and shared at the same time.

Copyright © 2010, Elsevier Inc. All rights Reserved

Clause examples

Trapezoidal rule

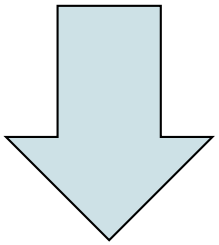
```
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++)
   approx += f(a + i*h);
approx = h*approx;
```



```
   h = (b-a)/n;
   approx = (f(a) + f(b))/2.0;
#  pragma omp parallel for num_threads(thread_count) \
      reduction(+: approx)
   for (i = 1; i <= n-1; i++)
      approx += f(a + i*h);
   approx = h*approx;
```

Estimating π

```
double factor = 1.0;
double sum = 0.0;
for (k = 0; k < n; k++) {
   sum += factor/(2*k+1);
   factor = -factor;
}
pi_approx = 4.0*sum;
```



```
      double sum = 0.0;
#     pragma omp parallel for num_threads(thread_count) \
         default(none) reduction(+:sum) private(k, factor) \
         shared(n)
      for (k = 0; k < n; k++) {
         if (k % 2 == 0)
            factor = 1.0;
         else
            factor = -1.0;
         sum += factor/(2*k+1);
      }
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Scoping clauses: first/lastprivate

- At the start of a parallel for, firstprivate initializes each thread's copy of a private variable to the value of the master's copy
  - Once per thread
  - Performance implications with respect to private
  - one-to-many

- At the end of a parallel for, lastprivate writes back to the master's copy the value contained in the private copy belonging to the thread that executed the "sequentially" last iteration of the loop
  - one-to-one (not many to one!)

Exercise: discuss *firstprivate* and *lastprivate* in "**parallel**" and "**parallel for**" cases

Lexical (static) and dynamic extent

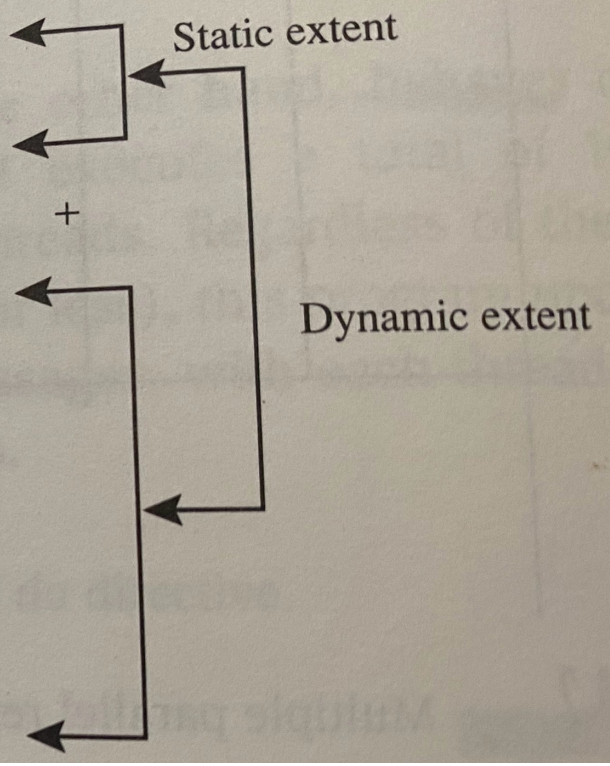
- We define the **lexical (or static) extent** of a parallel region as the code that is lexically within the parallel/end parallel directive

- We define **dynamic extent** of a parallel region to include not only the code that is directly between the parallel/end parallel directive (or static extent), but also to include all the code in subprograms that are invoked either directly or indirectly from within the parallel region.

Lexical and dynamic extent - Example

```
      program main
!$omp parallel
      call whoami
!$omp end parallel
      end

      subroutine whoami
      external omp_get_thread_num
      integer iam, omp_get_thread_num
      iam = omp_get_thread_num()
!$omp critical
      print *, "Hello from", iam
!$omp end critical
      return
      end
```



Exercise: default scoping rules in C

```
void caller(int a[], int n)
{
    int i, j, m = 3;

    #pragma omp parallel for
    for (i = 0; i < n; i++) {
        int k = m;

        for (j = 1; j ≤ 5; j++)
            callee(&a[i], &k, j);
    }
}

extern int c;

void callee(int *x, int *y, int z)
{
    int ii;
    static int cnt;

    cnt++;
    for (ii = 0; ii < z; i++)
        *x = *y + c;
}
```

Variable
a
n
i
j
m
k
x
**x*
y
**y*
z
c
ii
cnt

Take 5 minutes to complete the following exercise.

For each variable in the list, please provide:

- *Scope (shared/private)*
- *Reason for scope*
- *Is use safe? (Yes/No)*

Homework parallel/parallel for

- “Hello World” (parallel directive)
- Trapezoidal rule (parallel + critical)
- Trapezoidal rule (parallel for + reduction)
- Estimating π (parallel for + reduction)
- Try to run it on the cluster…

Copyright © 2010, Elsevier Inc. All rights Reserved

OpenMP Hands-on (I)

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{

 printf("Hello from the serial world\n");

#pragma omp parallel num_threads(10)
 {
  printf("Hello World... from thread = %d\n", omp_get_thread_num());
 }

 printf("Back to the serial world\n");
 return 0;
}
```

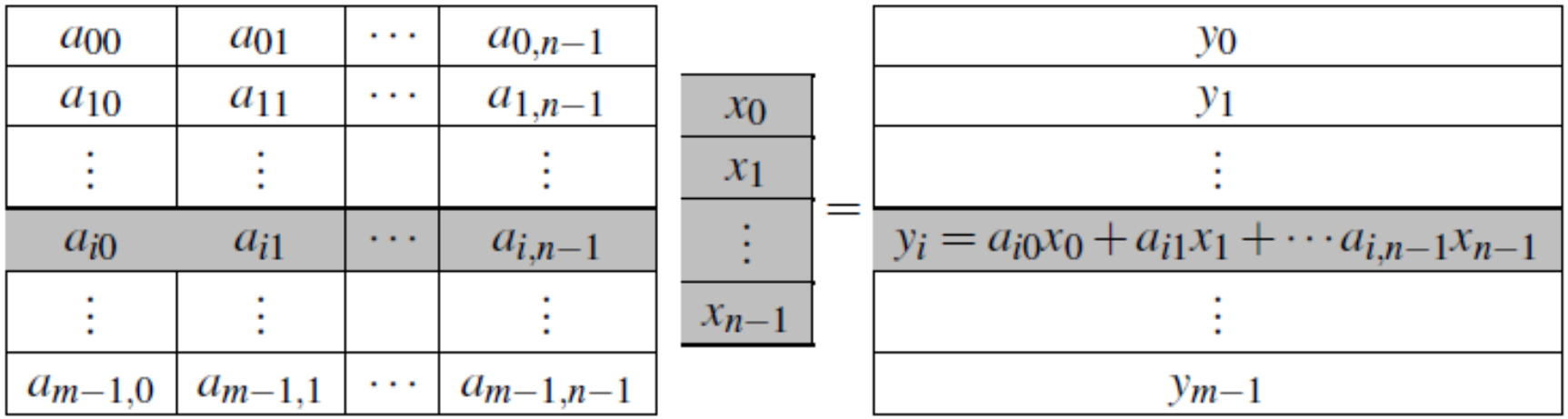
```
gcc -g  -Wall  -fopenmp  -o  omp_hello  omp_hello.c
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

OpenMP Hands-on (II)

Matrix vector multiplication



Trapezoidal rule

```
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++)
    approx += f(a + i*h);
approx = h*approx;
```

Estimating π

```
double factor = 1.0;
double sum = 0.0;
for (k = 0; k < n; k++) {
    sum += factor/(2*k+1);
    factor = -factor;
}
pi_approx = 4.0*sum;
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBI5A/viewform